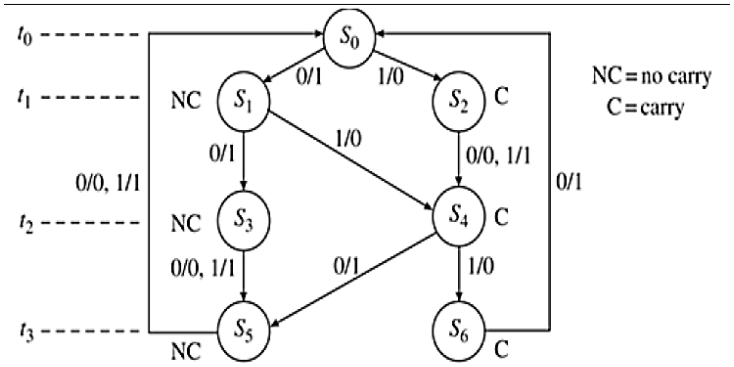
How to Create a BCD to Excess-3 State Diagram: Step-by-Step Guide

Understanding the Problem First

What we need to do: Convert BCD (Binary Coded Decimal) to Excess-3 code, processing one bit at a time from most significant bit (MSB) to least significant bit (LSB).

The Truth Table:

Decimal	BCD (4-bit)	Excess-3 (4-bit)
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100



Step 1: Understanding the Serial Addition Approach

Key Insight: Excess-3 = BCD + 3 (binary 0011)

Instead of converting directly, we can **add 3 to the BCD number bit by bit**, starting from the rightmost bit, just like manual addition!

Example: BCD 0101 (decimal 5) + 0011 (decimal 3)

0101 (BCD for 5)

+ 0011 (binary 3)

1000 (Excess-3 for 5)

Step 2: Why We Need States

When adding bit by bit from right to left, we need to remember:

- **Carry**: Did the previous addition generate a carry?
- **Position**: Which bit position are we currently processing?

This is why we need states! Each state represents:

- Which bit we're processing (t_0, t_1, t_2, t_3)
- Whether there's a carry from the previous position

Step 3: Analyzing Your State Diagram Structure

Time Slots (Bit Positions):

- **t_0**: Processing rightmost bit (LSB) - position 0
- **t_1**: Processing bit position 1
- **t_2**: Processing bit position 2
- **t_3**: Processing bit position 3 (MSB)

States in Your Diagram:

- **S_0**: Starting state, processing LSB, no carry
- **S_1**: Processing bit 1, no carry (NC)
- **S_2**: Processing bit 1, with carry (C)
- **S_3**: Processing bit 2, no carry (NC)
- **S_4**: Processing bit 2, with carry (C)
- **S_5**: Processing bit 3, no carry (NC)
- **S_6**: Processing bit 3, with carry (C)

Step 4: How the Addition Logic Works

Since we're adding 3 (binary 0011) to BCD:

- Position 0 (LSB): BCD bit + 1 (from 0011)
- Position 1: BCD bit + 1 (from 0011) + carry from position 0
- Position 2: BCD bit + 0 (from 0011) + carry from position 1

- Position 3 (MSB): BCD bit + 0 (from 0011) + carry from position 2

Step 5: Understanding the Transitions

Let me trace through your diagram:

At t_0 (LSB processing):

State S_0: Adding first bit of BCD + 1

- Input 0: 0+1 = 1, output 1, no carry → go to S_1 (NC)
- Input 1: 1+1 = 10_2, output 0, carry = 1 → go to S_2 (C)

At t_1:

From S_1 (no carry): Adding second bit + 1 + 0

- Input 0: 0+1+0 = 1, output 1, no carry → go to S_3 (NC)
- Input 1: 1+1+0 = 10_2, output 0, carry = 1 → go to S_4 (C)

From S_2 (with carry): Adding second bit + 1 + 1

- Input 0: 0+1+1 = 10_2, output 0, carry = 1 → go to S_4 (C)
- Input 1: 1+1+1 = 11_2, output 1, carry = 1 → go to S_4 (C)

At t_2:

From S_3 (no carry): Adding third bit + 0 + 0

- Input 0: 0+0+0 = 0, output 0, no carry → go to S_5 (NC)
- Input 1: 1+0+0 = 1, output 1, no carry → go to S_5 (NC)

From S_4 (with carry): Adding third bit + 0 + 1

- Input 0: 0+0+1 = 1, output 1, no carry → go to S_5 (NC)
- Input 1: 1+0+1 = 10_2, output 0, carry = 1 → go to S_6 (C)

At t_3 (MSB):

From S_5 (no carry): Adding fourth bit + 0 + 0

- Input 0: 0+0+0 = 0, output 0
- Input 1: 1+0+0 = 1, output 1

From S_6 (with carry): Adding fourth bit + 0 + 1

- Input 0: 0+0+1 = 1, output 1
- Input 1: 1+0+1 = 10_2, output 0 (carry discarded for 4-bit)

Step 6: Why This Design is Brilliant

1. **Systematic Approach**: Follows standard binary addition rules
2. **Memory Efficient**: Only needs to remember carry state
3. **Scalable**: Can extend to more bits easily
4. **Hardware Friendly**: Maps directly to flip-flops and logic gates
5. **Educational**: Shows clear relationship between addition and state machines

Step 7: Verification Example

Let's verify BCD 0101 (5) → Excess-3 1000 (8):

Processing 0101 (right to left):

t_0: Input=1, $S_0 \rightarrow S_2$, output=0

t_1: Input=0, $S_2 \rightarrow S_4$, output=0

t_2: Input=1, $S_4 \rightarrow S_5$, output=0

t_3: Input=0, $S_5 \rightarrow$ end, output=1

Result: 1000 ✓ (This is indeed Excess-3 for 5!)

This state diagram elegantly solves the BCD to Excess-3 conversion by treating it as a serial addition problem, making it both mathematically sound and practically implementable!